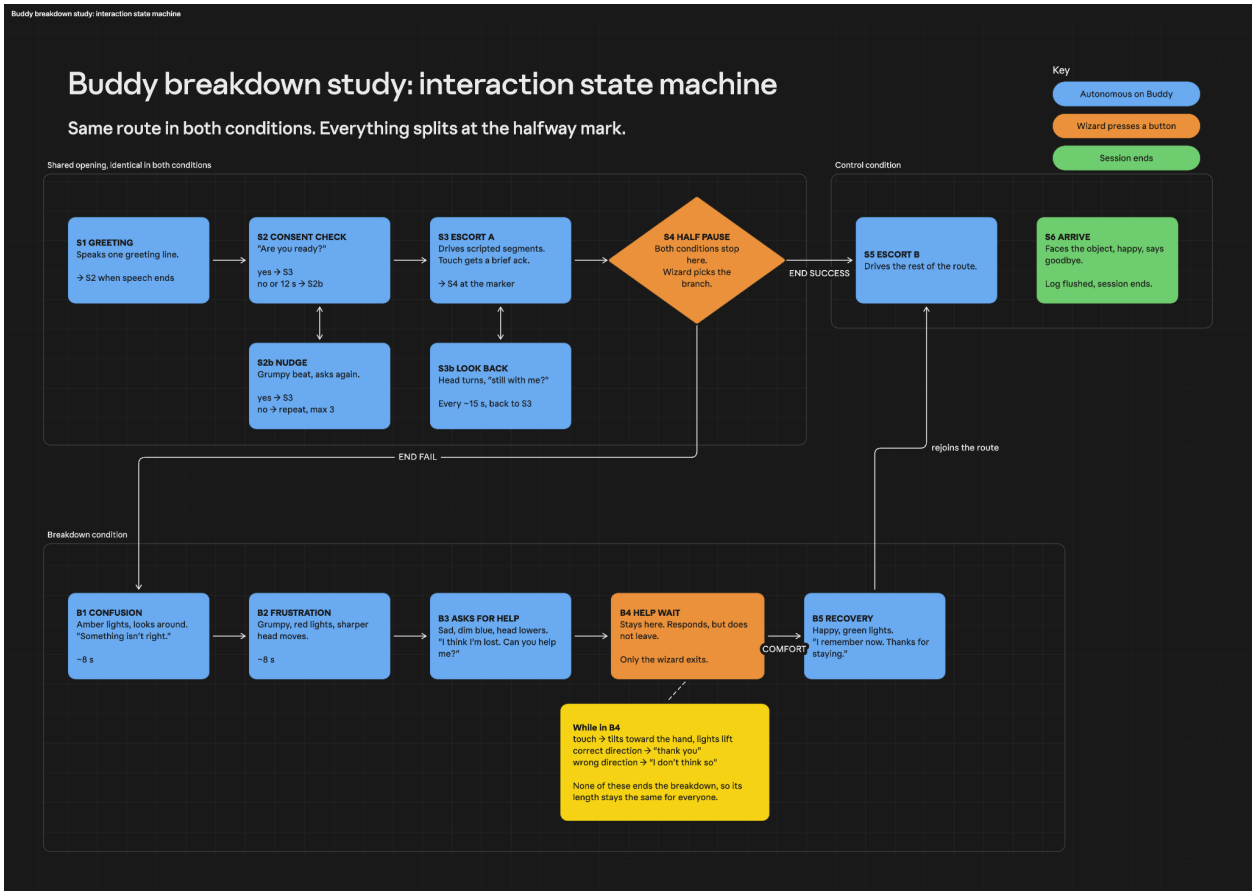
The architectural decision you need to make first



Split everything into two buckets, and be deliberate about which bucket each behaviour goes in.

Autonomous (runs on Buddy, deterministic, identical every session): movement segments, look-backs, touch response, the breakdown emotion sequence once triggered, all speech playback.

Wizard (a human presses a button): condition trigger at the halfway mark, yes/no recognition, whether a direction the participant gave was right or wrong, the comfort transition, abort.

Put **nothing** in the autonomous bucket that depends on understanding speech, and **nothing** in the wizard bucket that has to fire fast and consistently. That split is what keeps your two conditions comparable.

Answering your Q1 directly: no scanning, no SLAM

Script the route as a fixed sequence of motion segments and tape the floor. Reasons, in order of importance:

Mapping and autonomous navigation will not produce the same path twice. In a between-subjects design where the only intended difference is the breakdown, a route that varies run to run injects noise straight into your DV. Second, SLAM gives you a new class of failure right in the middle of a session, and you cannot tell the participant "ignore that, it's a real bug, the fake one comes later." Third, you have maybe three weeks.

So: a list of segments, each `{forward 2.4m}` or `{rotate -90°}`, executed in order, with a wizard "nudge left / nudge right / nudge forward" control for drift correction. Drift correction looks like Buddy adjusting himself, which is fine. Run the route ten times with no participant and tune the distances until it lands on the mark reliably.

Answering your Q3 and Q4 on listening: the wizard hears, not Buddy

Buddy has STT, but keyword spotting for "yes", "no", "left", "right" in a room with footsteps and a moving robot will misfire. Every misfire is a confound. The wizard is sitting there anyway. When the participant says yes, the wizard presses **YES** and Buddy responds in about 400ms, which reads as perfectly natural.

You can still run STT in parallel and log the transcript as data. Just don't let it drive the state machine.

The state machine

Condition (`CONTROL` / `BREAKDOWN`) is set in a config screen **before** the participant enters, not chosen during the run.

```
None
S0  BOOT              → setViewAsFace, neutral mood, wait for
W_START


S1  GREETING          speak: greeting variant
                      → auto to S2 on speech-complete


S2  CONSENT_CHECK     speak: "are you ready?" variant
                      on W_YES   → S3
                      on W_NO    → S2b
                      on 12s timeout → S2b
```

```
S2b NUDGE             mood GRUMPY briefly, speak: nudge variant
                      ("okay... what about now?")
                      on W_YES → S3
                      on W_NO  → S2b (max 3 nudges, then S3 anyway)

S3  ESCORT_A          execute segments[0..k]
                      every ~15s or at a tape marker → S3b
                      on TOUCH_HEAD → brief ack, stay in S3
                      on W_DISTRACTED → speak: concentrate variant,
stay in S3
                      → S4 when segment k completes (the halfway
marker)

S3b LOOK_BACK         rotate head ~160°, mood HAPPY,
                      speak: "still with me?" variant,
                      head returns, resume S3

S4  HALF_PAUSE        stop, neutral, LEDs steady
                      on W_END_SUCCESS   → S5   (control)
                      on W_END_FAIL      → S6   (breakdown)
                      on 10s timeout     → whichever matches the
configured condition
```

Control branch

None

```
S5  ESCORT_B          execute segments[k+1..n], same look-back rule
```

```
S5b ARRIVE            stop, face the object, mood HAPPY,
                      LEDs green pulse, speak: arrival + goodbye
variant
                      → S9 END
```

Breakdown branch

```
None
S6  CONFUSION         LEDs amber flicker, mood sceptical/neutral,
                      small left-right rotate (looking around),
                      speak: "hold on... something isn't right"
                      ~8s → S7

S7  FRUSTRATION       mood GRUMPY, LEDs red, sharper head motion,
                      speak: frustration variant
                      ~8s → S8

S8  SADNESS_APPEAL    mood SAD, LEDs dim blue, head lowers,
                      speak: "I think I'm lost... can you help me?"
                      → S8w

S8w HELP_WAIT         (the DV-rich state, stay here)
      on TOUCH_HEAD         → S8t, then back to S8w
      on W_DIR_CORRECT      → S8y, then back to S8w
      on W_DIR_WRONG        → S8n, then back to S8w
      on W_COMFORT          → S8r   ← only the wizard exits this
state
      on 120s timeout       → S8r
```

```
S8t TOUCH_ACK          head tilts toward the touched sensor,
                       LEDs brighten a little, soft vocalisation,
                       mood softens one step but does NOT reach
happy
                       ~3s → S8w

S8y GRATITUDE          mood HAPPY briefly, speak: "thank you"
variant
                       ~3s → S8w   (he does NOT start moving yet)

S8n GENTLE_DISAGREE    buddySayNo gesture, speak: "hmm, I don't
think
                       that's it" variant
                       ~3s → S8w

S8r RECOVERY           mood HAPPY, LEDs green, speak: "oh! I think I
                       remember now, thank you for staying"
                       → S5 (rejoins the control route to the
object)

S9  END                neutral face, stop all motion, flush log file
```

Three things in that machine that are deliberate, and why

Touch does not end the breakdown. You asked for touch to trigger "what Buddy normally does," and it does, but only as an acknowledgement. If touch exited the breakdown, then participants who touch would get a shorter breakdown than participants who don't, and the length of the distress episode would be entangled with the behaviour you're trying to measure. Only the wizard's button exits. Everyone gets roughly the same duration.

Gratitude does not resume movement. Same reason. If a correct direction made him drive off, then the breakdown would end earlier for participants who happen to guess right, and route-knowledge becomes a hidden variable.

The breakdown branch rejoins the route and reaches the object. If the control group finds the object and the breakdown group doesn't, then "did we succeed together" is confounded with "did he break down," and you can't tell which one moved the closeness score. Recovering and arriving late keeps the ending matched. Worth saying explicitly in the Methodology paragraph, because a reviewer will ask.

One more that you should decide, not me: your control condition currently has a "brief pause" where the breakdown has roughly 90 seconds of episode. Total interaction length then differs by condition, and longer exposure alone can raise closeness. You can either accept it and name it as a limitation, or pad the control pause with something neutral ("let me just check the route") to narrow the gap. I'd pad it. It's cheap.

Speech bank

Keep variants **matched in content and warmth**, varying only surface wording. If one greeting variant is warmer than another you've added noise to your DV for free. Log which variant index fired, every time.

```
Java
GREETING      = ["Hi there, I'm Buddy. I'll take you to
something.",
                 "Hello! I'm Buddy. There's something I want to
show you.",
                 "Hey, I'm Buddy. Follow me and I'll bring you to
it."]


READY_Q       = ["Are you ready?", "Shall we go?", "Ready when you
are."]


NUDGE         = ["Okay... what about now?",
                 "Alright. Now?",
                 "Hmm. How about now?"]
```

```
LOOK_BACK     = ["Still with me?", "Are you following?", "You
there?"]

CONCENTRATE   = ["One moment, let me concentrate.",
                 "Hold that thought, I need to focus.",
                 "Give me a second, I'm concentrating."]

CONFUSED      = ["Hold on... something isn't right.",
                 "Wait. This doesn't look right."]

FRUSTRATED    = ["This isn't working. It should be here.",
                 "No, no. It was supposed to be this way."]

LOST_APPEAL   = ["I think I'm lost. Can you help me?",
                 "I don't know where to go. Will you help me?"]

THANKS        = ["Thank you.", "Oh, thank you.", "That helps, thank
you."]

DISAGREE      = ["Hmm, I don't think that's it.",
                 "I don't think so, not that way."]

RECOVER       = ["Oh! I think I remember now. Thank you for
staying.",
                 "Wait, yes. I've got it. Thanks for not leaving."]
```

```
ARRIVE        = ["Here it is! That's what I wanted to show you.
Goodbye!",
                 "There. We made it. Thanks for coming with me,
goodbye!"]
```

On your "don't distract me" line: I softened it to "let me concentrate." The blunter version reads as rude and will drag likeability down in both conditions, which doesn't break the comparison but does compress your Godspeed range. Your call.

The wizard panel

You need a way to press buttons without the participant seeing you do it. Cleanest option given you're a Python person: embed **NanoHTTPD** in the Android app (one dependency, about 60 lines), serve a single HTML page with big buttons, and open `http://192.168.1.214:8080` in a browser on your laptop or phone from across the room. Each button is a GET that posts an event into the state machine's queue. No client code to write at all.

Buttons you need:

```
None
START          YES          NO          NUDGE

DISTRACTED     TOUCH-SIM

END SUCCESS    END FAIL

DIR CORRECT    DIR WRONG    COMFORT

NUDGE L  NUDGE R  NUDGE FWD     ABORT
```

Grey out the buttons that are invalid in the current state and show the current state name at the top. Under pressure the wizard will fumble, and a panel that only lets you press legal things saves sessions.

Logging

Write one CSV per session on Buddy's internal storage, pull it with `adb pull`. One row per event: `timestamp, participant_id, condition, state,`

`event_source(auto|wizard|sensor), event, utterance_variant_index`. That file gives you time-in-help-state, touch count and latency, number of direction attempts, and whether they stayed, which is exactly the behavioural data in your Methodology paragraph. Set that up on day one, not the night before the first session.

Build order

Get the smoke test running. Then the motion segment runner with the nudge controls, and tune the route on the taped floor. Then the HTTP panel and the event queue. Then the state machine with placeholder speech. Then the real speech bank and the emotion sequence. Then logging. Then two full dry runs with a groupmate playing participant, one per condition.

The piece most likely to eat your time is the route tuning, not the code. Start that as soon as you have motion working.

Want me to put the state machine on a Whiteboard so your group can see it and move pieces around?